

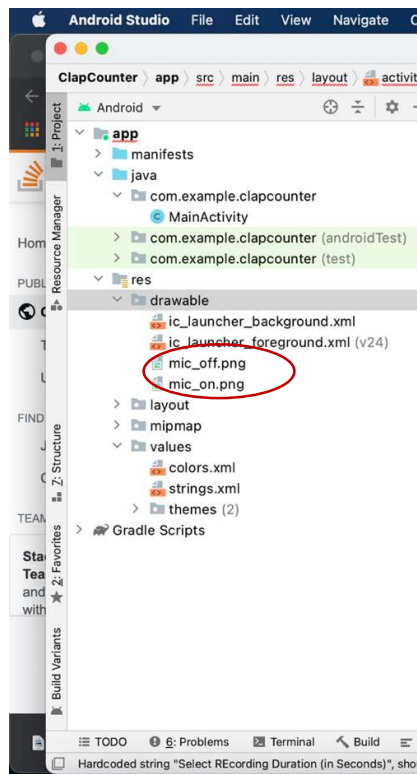


Tut-12 Clap counter APP using Audio Recorder Sensor Programming and Android Studio(21B12CI312) 6th Sem Elective, CSE/IT

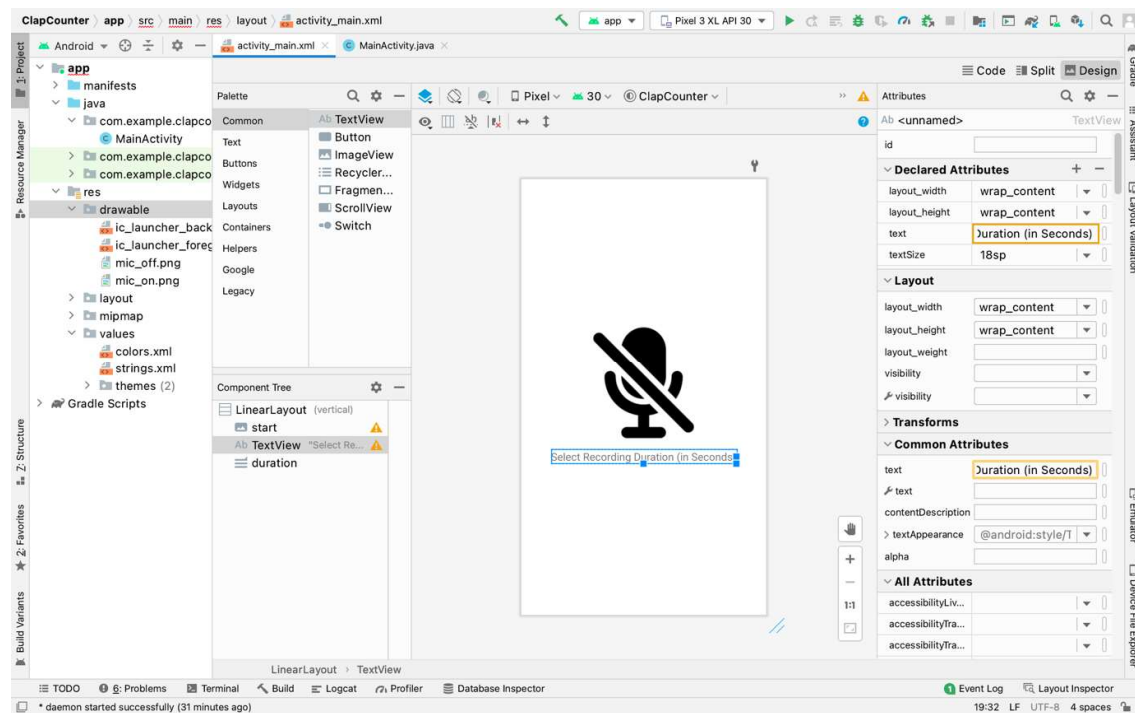
1. Create an Empty Activity Project with name of your choice



2. Add mic_on and mic_off in drawable by copy and paste



3. Design the GUI with Linear layout with Image Button, Text View and Spinner as shown below



Activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainActivity"
    android:orientation="vertical"
    android:gravity="center">

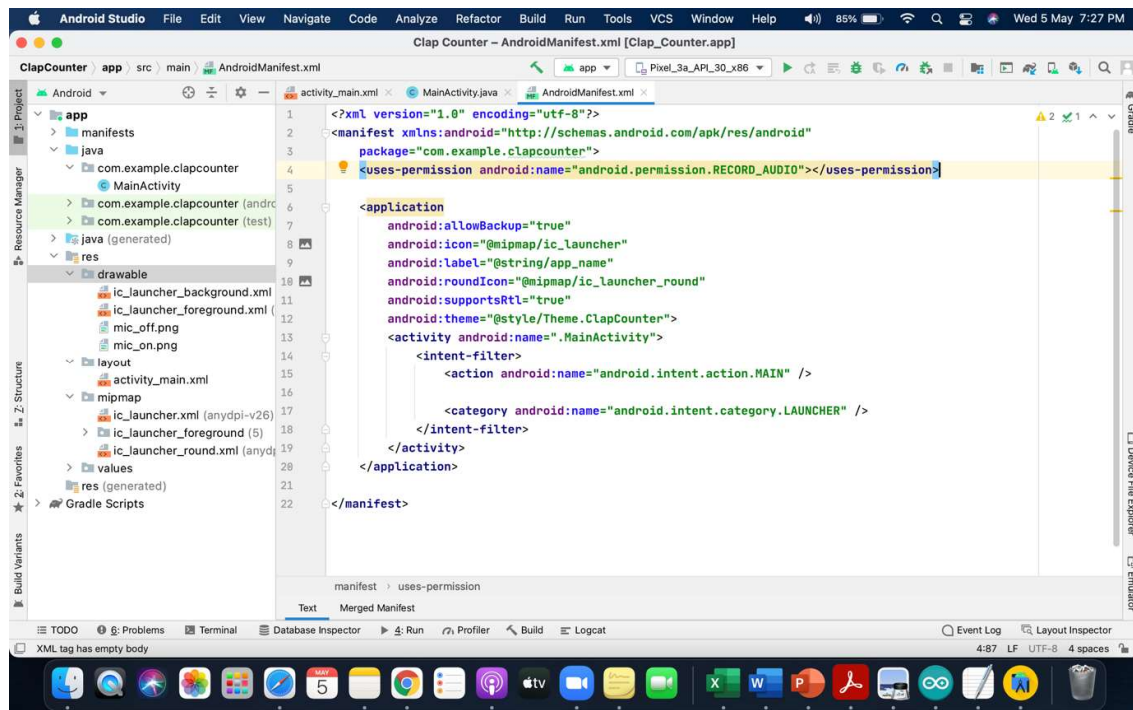
    <ImageButton
        android:id="@+id/start"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:background="@drawable/mic_off"/>

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Select Recording Duration (in Seconds)"
        android:textSize="18sp" />

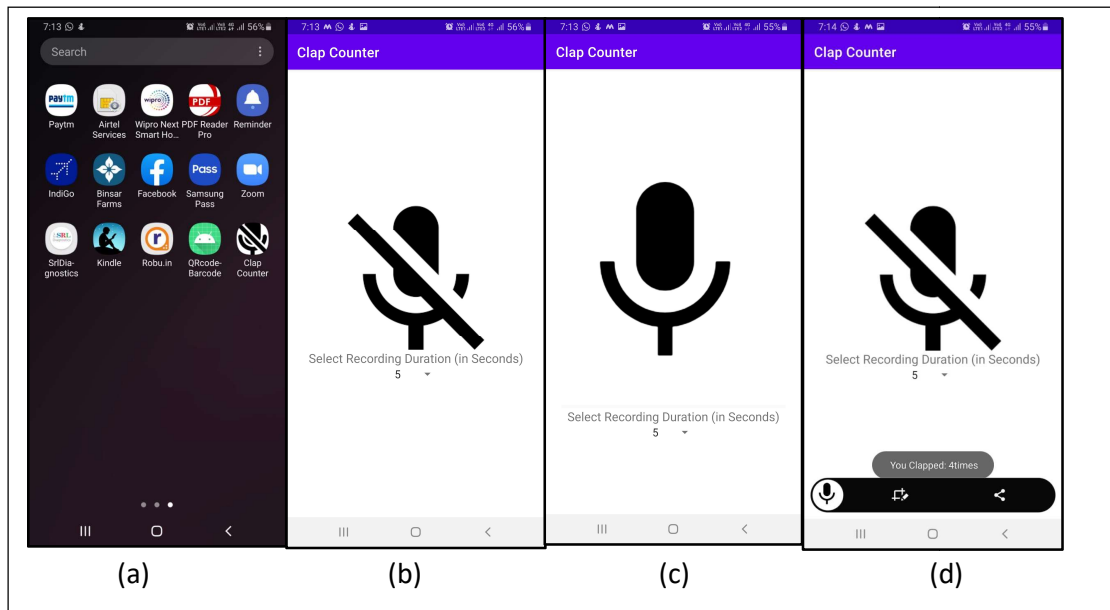
    <Spinner
        android:id="@+id/duration"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"/>

</LinearLayout>
```

4. Add user permission for Recording Audio in manifest file



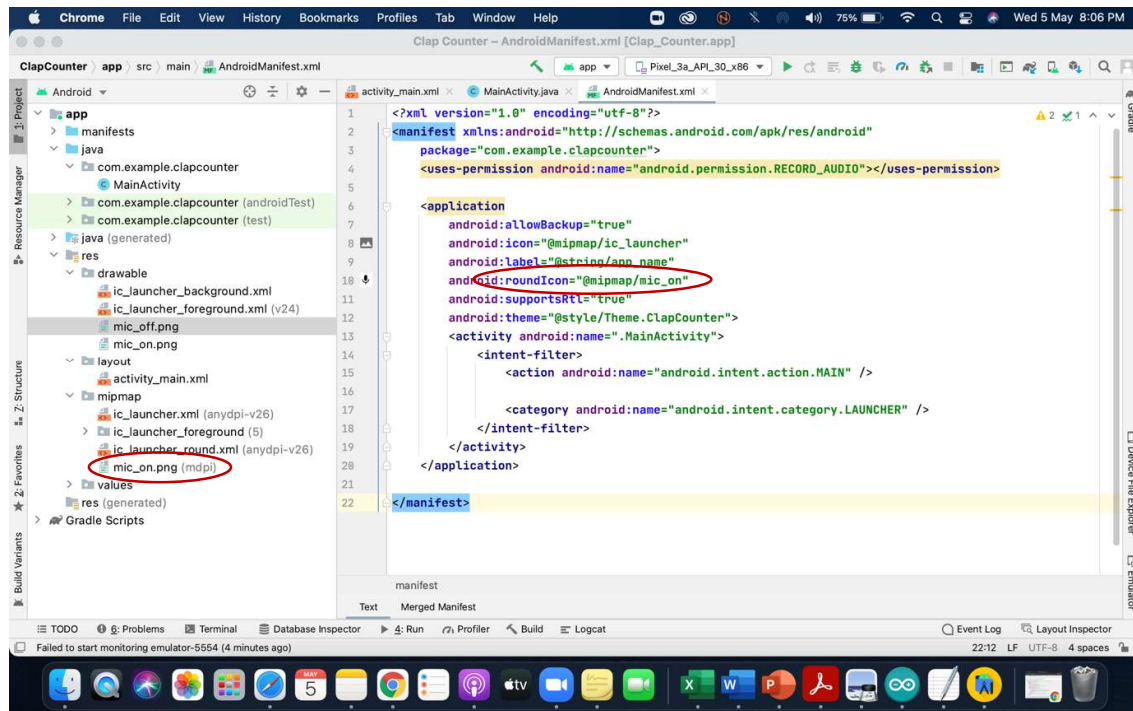
5. Now let us program for the following functionality



- (a) Icon of the clap count app
- (b) App open view, select the duration as 5 sec, 10 sec, 15 sec or 20 sec and click on mic_off symbol to turn mic_on
- (c) Make the clap sound repeatedly to record the count
- (d) After selected time lapses the count will be displayed back on the original screen

5 (a) Icon for the clap count app

To set the icon of the Clap Count APP, add mic_on.png in mipmap and rename roundIcon as shown [1]

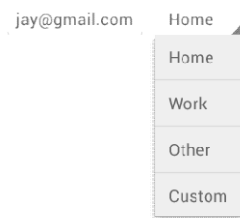


5(b)

i) Spinner Drop Down for duration selection

<https://developer.android.com/guide/topics/ui/controls/spinner#java>

Spinners provide a quick way to select one value from a set. In the default state, a spinner shows its currently selected value. Touching the spinner displays a dropdown menu with all other available values, from which the user can select a new one.



In xml file we have already added

```
<Spinner
    android:id="@+id/planets_spinner"
    android:layout_width="match_parent"
    android:layout_height="wrap_content" />
```

Key classes are the following:

- [Spinner](#)
- [SpinnerAdapter](#)
- [AdapterView.OnItemSelectedListener](#)

Syntax for Selection based on the Array of choice

```
Spinner spinner = (Spinner) findViewById(R.id.spinner);
// Create an ArrayAdapter using the string array and a default spinner layout
ArrayAdapter<CharSequence> adapter =
ArrayAdapter.createFromResource(this,
    R.array.planets_array, android.R.layout.simple_spinner_item);
// Specify the layout to use when the list of choices appears
adapter.setDropDownViewResource(android.R.layout.simple_spinner_dropdown_item);
// Apply the adapter to the spinner
spinner.setAdapter(adapter);
```

ii) Media Recording

<https://developer.android.com/guide/topics/media/mediarecorder>

MediaRecorder overview

The Android multimedia framework includes support for capturing and encoding a variety of common audio and video formats. You can use the [MediaRecorder](#) APIs if supported by the device hardware.

Requesting permission to record audio

```
<uses-permission android:name="android.permission.RECORD_AUDIO"/>
```

Creating and running a MediaRecorder

Initialize a new instance of [MediaRecorder](#) with the following calls:

- Set the audio source using [setAudioSource\(\)](#). You'll probably use [MIC](#).

Note: Most of the audio sources (including `DEFAULT`) apply processing to the audio signal. To record raw audio select `UNPROCESSED`. Some devices do not support unprocessed input.

Call `AudioManager.getProperty(AudioManager.PROPERTY_SUPPORT_AUDIO_SOURCE_UNPROCESSED)` first to verify it's available. If it is not, try using `VOICE_RECOGNITION` instead, which does not employ AGC or noise suppression. You can use `UNPROCESSED` as an

audio source even when the property is not supported, but there is no guarantee whether the signal will be unprocessed or not in that case.

- Set the output file format using `setOutputFormat()`. Note that starting with Android 8.0 (API level 26) `MediaRecorder` supports the MPEG2_TS format, which is useful for streaming:

KOTLINJAVA

```
mediaRecorder.setOutputFormat(MediaRecorder.OutputFormat.MPEG_2_TS);
```

- Set the output file name using `setOutputFile()`. You must specify a file descriptor that represents an actual file.
- Set the audio encoder using `setAudioEncoder()`.
- Complete the initialization by calling `prepare()`.

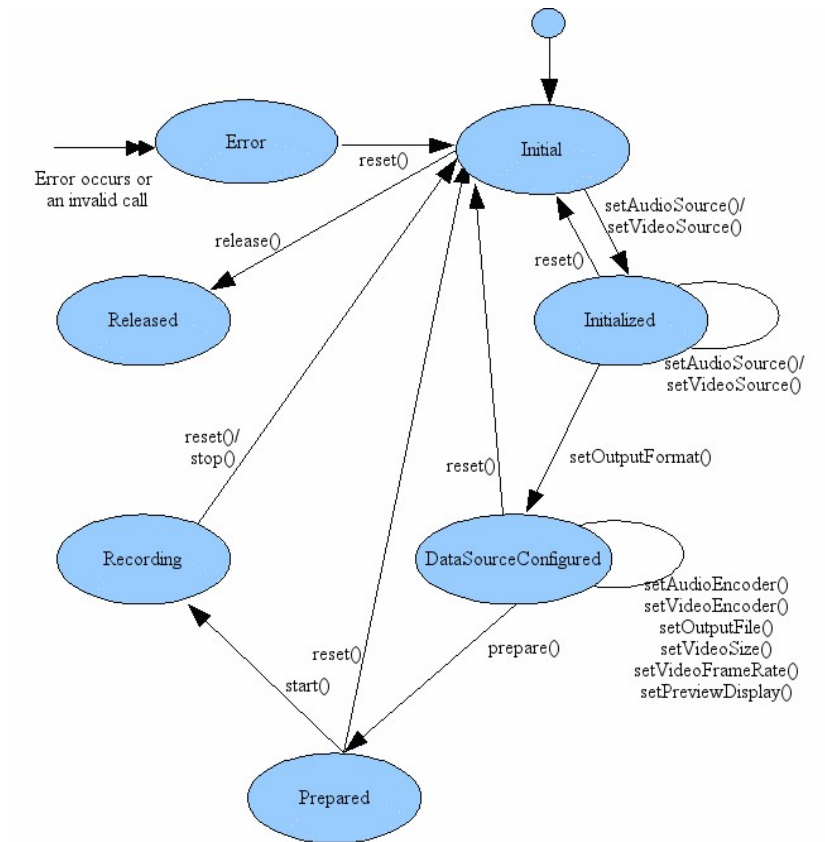
Start and stop the recorder by calling `start()` and `stop()` respectively.

When you are done with the `MediaRecorder` instance free its resources as soon as possible by calling `release()`.

Note: On devices running Android 9 (API level 28) or higher, apps running in the background cannot access the microphone. Therefore, your app should record audio only when it's in the foreground or when you include an instance of `MediaRecorder` in a foreground service.

<https://developer.android.com/reference/android/media/MediaRecorder>

Used to record audio and video. The recording control is based on a simple state machine (see below).

**MediaRecorder state diagram**

A common case of using MediaRecorder to record audio works as follows:

```

MediaRecorder recorder = new MediaRecorder();
recorder.setAudioSource(MediaRecorder.AudioSource.MIC);
recorder.setOutputFormat(MediaRecorder.OutputFormat.THREE_GPP);
recorder.setAudioEncoder(MediaRecorder.AudioEncoder.AMR_NB);
recorder.setOutputFile(PATH_NAME);
recorder.prepare();
recorder.start();    // Recording is now started
...
recorder.stop();
recorder.reset();    // You can reuse the object by going back to
                    // setAudioSource() step
recorder.release();  // Now the object cannot be reused

```

Clap App Code:

```
package com.example.clapcounter;
```

```

import androidx.appcompat.app.AppCompatActivity;
import androidx.core.app.ActivityCompat;

import android.Manifest;
import android.content.pm.PackageManager;
import android.media.MediaRecorder;
import android.os.AsyncTask;
import android.os.Bundle;
import android.view.View;
import android.widget.ArrayAdapter;
import android.widget.ImageButton;
import android.widget.Spinner;
import android.widget.Toast;

import java.io.IOException;

public class MainActivity extends AppCompatActivity {
    final int RECORD_AUDIO = 0;
    ImageButton btnStart;
    Spinner spinDuration;
    int clipDuration=0;
    LongOperation recordAudioSync= null;
    String[] durationItems= {"5", "10", "15", "20"};

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        initializeHandles();
        ArrayAdapter<String> spinDurationAdapter = new ArrayAdapter<>(this,
            android.R.layout.simple_spinner_item, durationItems);

        spinDurationAdapter.setDropDownViewResource(android.R.layout.simple_spinner
            _dropdown_item);
        spinDuration.setAdapter(spinDurationAdapter);
        btnStart.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View v) {
                clipDuration= Integer.parseInt(spinDuration.getSelectedItem().toString());
                if (ActivityCompat.checkSelfPermission(MainActivity.this,
                    Manifest.permission.RECORD_AUDIO) != PackageManager.PERMISSION_GRANTED)
                {

                    ActivityCompat.requestPermissions(MainActivity.this, new String[]{
                        Manifest.permission.RECORD_AUDIO}, RECORD_AUDIO);
                } else {
                    recordAudioSync= new LongOperation();
                    recordAudioSync.execute("");
                }
            }
        });

        private void initializeHandles(){
            btnStart=findViewById(R.id.start);
            spinDuration=findViewById(R.id.duration);
        }

    @Override
    protected void onDestroy() {
        super.onDestroy();
    }

```

```

if(recordAudioSync!= null &&recordAudioSync.getStatus() !=
AsyncTask.Status.FINISHED) {
    recordAudioSync.done();
    recordAudioSync.cancel(true);

    }

}

@Override
protected void onPause() {
    super.onPause();
    if(recordAudioSync!= null &&recordAudioSync.getStatus() !=
AsyncTask.Status.FINISHED) {
        recordAudioSync.done();
        recordAudioSync.cancel(true);

    }

}

private class LongOperation extends AsyncTask< String, Void, String > {

    MediaRecorder recorder;
    int ClapDetectedNumber;

    @Override
    protected String doInBackground(String... strings) {
        recordAudio();
        return "" + ClapDetectedNumber;
    }

    @Override
    protected void onPreExecute() {
        btnStart.setImageResource(R.drawable.mic_on);
        btnStart.setEnabled(false);

    }

    @Override
    protected void onPostExecute(String s) {
        Toast.makeText(MainActivity.this, " You Clapped: " + s + "times",
        Toast.LENGTH_SHORT).show();
        btnStart.setImageResource(R.drawable.mic_off);
        btnStart.setEnabled(true);
    }

    private void recordAudio() {
        recorder = new MediaRecorder();
        recorder.setAudioSource(MediaRecorder.AudioSource.MIC);
        recorder.setOutputFormat(MediaRecorder.OutputFormat.THREE_GPP);
        recorder.setAudioEncoder(MediaRecorder.AudioEncoder.AMR_NB);
        recorder.setOutputFile("/data/data/" + getPackageName()+ "/music.3gp");
        int startAmplitude = 0;
        int finishAmplitude;
        int amplitudeThreshold = 18000;
        int counter =0 ;
        try {
            recorder.prepare();
            recorder.start();
            startAmplitude = recorder.getMaxAmplitude();

```

```

        } catch (IOException e) {
e.printStackTrace();
        }
do {
if(isCancelled()){
break;
        }
        counter++;
waitSome();
finishAmplitude = recorder.getMaxAmplitude();
if(finishAmplitude>= amplitudeThreshold) {
ClapDetectedNumber++;
        }
}while(counter < (clipDuration* 4));
done();

    }

private void waitSome() {
try {
Thread.sleep(250);
    } catch (InterruptedException e) {
e.printStackTrace();
    }
}

private void done() {
if(recorder != null) {
recorder.stop();
recorder.release();
    }
}
}
}

```

Reference:

1. <https://www.tutorialspoint.com/how-can-i-set-an-icon-for-my-android-application#:~:text=Step%20%E2%88%92%20Create%20a%20new,the%20image%20in%20mipmap%20folder.>
2. <https://developer.android.com/guide/topics/ui/controls/spinner#java>
3. <https://developer.android.com/guide/topics/media/mediarecorder>
4. <https://developer.android.com/reference/android/media/MediaRecorder>